```
/// Blood through arteries (LAD) artery analysis
// The flow is assumed to be unsteady, incompressible, laminar, Newtonian, and fully developed

#include <stdio.h>
#include <math.h>
#include <stdlib.h>
#include <cuda.h>
#include <cuda_runtime.h>

// Simulation parameters
#define rho 1.06e3f
#define L  0.14f
#define W  0.0035f
#define U  0.39f
#define V  0.0f
#define DX 0.0001f
#define DY 0.0001f
#define dt 1.0e-5f
#define NX ((int)(L/DX)+1)

#define NY ((int)(W/DY)+1)
#define N  (NX*NY)
#define del_P 2.8863e3f
#define mu0    0.056f
#define mu_inf 0.0035f
#define lambda 3.13f
#define n      0.6f
#define a      2.0f

// Function prototypes for kernels
__global__ void Initialize(float *u, float *v, float *p, float *visc, float *shear_rate, float p_init, int nx, int ny);
__global__ void ApplyBoundaryConditions(float *u, float *v, float *p, float p_out, int nx, int ny);
__global__ void Shear_rate(float *u, float *v, float *mu, float *shear_rate, int nx, int ny);
__global__ void Xmomentum_solver(float *u_x, float *u_y, float *p, float *mu, float *u_x_new, int nx, int ny);
__global__ void YMomentum_solver(float* u_x, float* u_y, float* p, float* v_new, float dt, float dx, float dy, float rho, float* mu, int nx, int ny);
__global__ void Divergence(float *u_x_new, float *v_new, float *divergence, int nx, int ny);
__global__ void PoissonSolverGS(float *divergence, float *p, float *p_new, int nx, int ny, float omega, float *residual);
__global__ void CorrectVelocities(float *u, float *v, float *u_star, float *v_star, float *p, int nx, int ny);
__global__ void CalculateResidual(float *u, float *u_old, float *v, float *v_old, float *residual, int nx, int ny);
__global__ void CalculateMaxVelocity(float *u, float *v, float *max_velocity, int nx, int ny);
void saveVTKFile(float *u, float *v, float *p, float *mu, float *shear_rate, int nx, int ny, int timestep);
bool SolvePressurePoisson(float *d_divergence, float *d_p, float *d_p_new, int nx, int ny, float omega, float tolerance);


//initializing the variables for the simulation
// Device arrays for staggered grid
__device__ float u[(NX+1)*NY];     // x-velocity at east/west cell faces
```

```
__device__ float v[NX*(NY+1)];     // y-velocity at north/south cell faces
__device__ float p[NX*NY];         // pressure at cell centers
__device__ float visc[NX*NY];      // viscosity at cell centers
__device__ float shear_rate[NX*NY]; // shear rate at cell centers

// Arrays for SIMPLE algorithm
__device__ float u_star[(NX+1)*NY];  // intermediate x-velocity
__device__ float v_star[NX*(NY+1)];  // intermediate y-velocity
__device__ float p_prime[NX*NY];     // pressure correction
__device__ float u_old[(NX+1)*NY];   // previous iteration x-velocity
__device__ float v_old[NX*(NY+1)];   // previous iteration y-velocity

// Host arrays
float *h_u, *h_v, *h_p, *h_visc, *h_shear_rate;

// Allocate host memory
h_u = (float*)malloc(sizeof(float) * (NX+1) * NY);
h_v = (float*)malloc(sizeof(float) * NX * (NY+1));
h_p = (float*)malloc(sizeof(float) * NX * NY);
h_visc = (float*)malloc(sizeof(float) * NX * NY);
h_shear_rate = (float*)malloc(sizeof(float) * NX * NY);

// Device pointers
float *d_u, *d_v, *d_p, *d_visc, *d_shear_rate;
float *d_u_star, *d_v_star, *d_p_prime;

// Allocate device memory
cudaMalloc((void**)&d_u, sizeof(float) * (NX+1) * NY);
cudaMalloc((void**)&d_v, sizeof(float) * NX * (NY+1));
cudaMalloc((void**)&d_p, sizeof(float) * NX * NY);
cudaMalloc((void**)&d_visc, sizeof(float) * NX * NY);
cudaMalloc((void**)&d_shear_rate, sizeof(float) * NX * NY);
cudaMalloc((void**)&d_u_star, sizeof(float) * (NX+1) * NY);
cudaMalloc((void**)&d_v_star, sizeof(float) * NX * (NY+1));
cudaMalloc((void**)&d_p_prime, sizeof(float) * NX * NY);

// Initialize host arrays
// Initialize pressure (cell centers)
for (int j = 0; j < NY; j++) {
    for (int i = 0; i < NX; i++) {
        int idx = i + j*NX;
        // Linear pressure drop from inlet to outlet
        h_p[idx] = del_P * (1.0f - (float)i/NX);
        // Initialize viscosity to zero-shear value
        h_visc[idx] = mu0;
        // Initialize shear rate to zero
        h_shear_rate[idx] = 0.0f;
    }
}

// Initialize x-velocity (east/west cell faces)
for (int j = 0; j < NY; j++) {
    for (int i = 0; i <= NX; i++) {
        int idx = i + j*(NX+1);
        h_u[idx] = 0.0f;

        // Set inlet boundary condition (west boundary, i=0)
        if (i == 0) {
            // Parabolic profile at inlet
            float y_rel = (float)j/(NY-1) - 0.5f;  // -0.5 to 0.5
```

```
            h_u[idx] = U * (1.0f - 4.0f * y_rel * y_rel);  // Parabolic profile
        }
    }
}

// Initialize y-velocity (north/south cell faces)
for (int j = 0; j <= NY; j++) {
    for (int i = 0; i < NX; i++) {
        int idx = i + j*NX;
        h_v[idx] = 0.0f;
    }
}

// Copy initial data from host to device
cudaMemcpy(d_p, h_p, sizeof(float)*NX*NY, cudaMemcpyHostToDevice);
cudaMemcpy(d_visc, h_visc, sizeof(float)*NX*NY, cudaMemcpyHostToDevice);
cudaMemcpy(d_shear_rate, h_shear_rate, sizeof(float)*NX*NY, cudaMemcpyHostToDevice);
cudaMemcpy(d_u, h_u, sizeof(float)*(NX+1)*NY, cudaMemcpyHostToDevice);
cudaMemcpy(d_v, h_v, sizeof(float)*NX*(NY+1), cudaMemcpyHostToDevice);

/// Initialize flow field with initial conditions
__global__ void Initialize(float *u, float *v, float *p, float *visc, float *shear_rate,
float p_init, int nx, int ny) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    // Initialize pressure and viscosity at cell centers
    if (i < nx && j < ny) {
        int idx = i + j*nx;  // Cell-centered index
        p[idx] = p_init * (1.0f - (float)i/nx);  // Linear pressure gradient
        visc[idx] = mu0;  // Initialize with zero-shear viscosity
        shear_rate[idx] = 0.0f;  // Initialize shear rate to zero
    }

    // Initialize x-velocity at east/west faces (staggered grid)
    if (i <= nx && j < ny) {  // Note: i can be nx for the last face
        int idx_u = i + j*(nx+1);  // Face-centered index for u

        if (i == 0) {
            // Inlet boundary condition - uniform flow for non-pulsatile
            u[idx_u] = U;
        } else if (i == nx) {
            // Outlet boundary condition
            u[idx_u] = u[idx_u-1];  // Zero gradient
        } else {
            // Interior points
            u[idx_u] = 0.0f;  // Initialize to zero
        }
    }

    // Initialize y-velocity at north/south faces (staggered grid)
    if (i < nx && j <= ny) {  // Note: j can be ny for the last face
        int idx_v = i + j*nx;  // Face-centered index for v
        v[idx_v] = 0.0f;  // Initialize to zero

        // No-slip boundary conditions at walls (north/south)
        if (j == 0 || j == ny) {
            v[idx_v] = 0.0f;
        }
    }
```

```
}

//Boundary conditions for the flow field
// Apply boundary conditions for staggered grid
__global__ void ApplyBoundaryConditions(float *u, float *v, float *p, float p_out, int nx, int ny) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    // Inlet boundary (west face, x = 0)
    if (i == 0 && j < ny) {
        // u-velocity at inlet face (i=0)
        u[j*(nx+1) + i] = U;  // Uniform inlet velocity

        // v-velocity near inlet
        if (j < ny) {
            v[j*nx + i] = 0.0f;  // No y-component at inlet
        }

        // Pressure at inlet cells
        p[j*nx + i] = p[j*nx + (i+1)];  // Zero gradient
    }

    // Outlet boundary (east face, x = nx)
    if (i == nx && j < ny) {
        // u-velocity at outlet face (i=nx)
        u[j*(nx+1) + i] = u[j*(nx+1) + (i-1)];  // Zero gradient

        // Pressure at outlet cells (i=nx-1)
        if (i-1 >= 0) {
            p[j*nx + (i-1)] = p_out;  // Fixed pressure at outlet
        }
    }

    // Wall boundaries (south face, y = 0)
    if (j == 0 && i < nx) {
        // v-velocity at bottom wall (j=0)
        v[j*nx + i] = 0.0f;  // No-slip condition

        // u-velocity near bottom wall
        if (i < nx+1) {
            u[j*(nx+1) + i] = 0.0f;  // No-slip condition
        }
    }

    // Wall boundaries (north face, y = ny)
    if (j == ny && i < nx) {
        // v-velocity at top wall (j=ny)
        v[j*nx + i] = 0.0f;  // No-slip condition

        // u-velocity near top wall
        if (i < nx+1) {
            u[(j-1)*(nx+1) + i] = 0.0f;  // No-slip condition
        }
    }
}

// Calculate shear rate and update viscosity using Bird-Carreau model
__global__ void Shear_rate(float *u, float *v, float *mu, float *shear_rate, int nx, int ny) {
```

```
225     int i = blockIdx.x * blockDim.x + threadIdx.x;
226     int j = blockIdx.y * blockDim.y + threadIdx.y;
227 
228     if (i > 0 && i < nx-1 && j > 0 && j < ny-1) {
229         // For staggered grid, u is at (i+1/2,j) and v is at (i,j+1/2)
230         // Calculate velocity gradients at cell center (i,j)
231         float du_dx = (u[j*(nx+1) + (i+1)] - u[j*(nx+1) + i]) / DX;
232         float dv_dy = (v[(j+1)*nx + i] - v[j*nx + i]) / DY;
233 
234         // Cross derivatives require averaging
235         float du_dy = (u[(j+1)*(nx+1) + i] + u[(j+1)*(nx+1) + (i+1)] -u[j*(nx+1) + i] -
    u[j*(nx+1) + (i+1)]) / (2.0f * DY);
236         float dv_dx = (v[j*nx + (i+1)] + v[(j+1)*nx + (i+1)] -v[j*nx + i] - v[(j+1)*nx +
    i]) / (2.0f * DX);
237 
238         // Calculate shear rate
239         float sr = sqrtf(2.0f * (du_dx*du_dx + dv_dy*dv_dy) + (du_dy + dv_dx)*(du_dy +
    dv_dx));
240 
241         // Store at cell center
242         int idx = j*nx + i;
243         shear_rate[idx] = sr;
244 
245         // Update viscosity using Bird-Carreau model
246         mu[idx] = mu_inf + (mu0 - mu_inf) * powf(1.0f + powf(lambda * sr, a), (n-1.0f)/a);
247     } else if (i < nx && j < ny) {
248         // Set boundary values to zero or a safe default
249         int idx = j*nx + i;
250         shear_rate[idx] = 0.0f;
251         mu[idx] = mu0;
252     }
253 }
254 
255 //////////*************************SIMPLE ALGORITHM
    STARTED*********************************//////////////////
256 
257 //solver for x momentum equation
258 //Xmomentum equation solver
259 __global__ void Xmomentum_solver(float *u_x, float *u_y, float *p, float *mu, float
    *u_x_new, int nx, int ny) {
260     int i = blockIdx.x * blockDim.x + threadIdx.x;
261     int j = blockIdx.y * blockDim.y + threadIdx.y;
262 
263     if (i >= 1 && i < nx-1 && j >= 1 && j < ny-1) {
264         // For staggered grid, u_x is stored at (i+1/2, j)
265         // Proper indexing for u_x: j*(nx+1) + i
266         int idx_u = j*(nx+1) + i;
267 
268         // 1. Calculate convection term using QUICK scheme
269         // Calculate mass fluxes through cell faces
270         float Fe = 0.5f * (u_x[j*(nx+1) + i] + u_x[j*(nx+1) + (i+1)]) * DY;
271         float Fw = 0.5f * (u_x[j*(nx+1) + (i-1)] + u_x[j*(nx+1) + i]) * DY;
272 
273         // Interpolate v-velocities to u-cell faces
274         float vn = 0.25f * (u_y[(j+1)*nx + i] + u_y[(j+1)*nx + (i-1)] +u_y[j*nx + i] +
    u_y[j*nx + (i-1)]);
275         float vs = 0.25f * (u_y[j*nx + i] + u_y[j*nx + (i-1)] +u_y[(j-1)*nx + i] +
    u_y[(j-1)*nx + (i-1)]);
276 
277         float Fn = vn * DX;
```

```
278         float Fs = vs * DX;
279 
280         // QUICK scheme for face values
281         float phi_e, phi_w, phi_n, phi_s;
282 
283         // East face
284         if (Fe >= 0) {
285             phi_e = (3.0f/8.0f) * u_x[j*(nx+1) + (i+1)] +(6.0f/8.0f) * u_x[j*(nx+1) + i] -
    (1.0f/8.0f) * u_x[j*(nx+1) + (i-1)];
286         } else {
287             phi_e = (3.0f/8.0f) * u_x[j*(nx+1) + i] +(6.0f/8.0f) * u_x[j*(nx+1) + (i+1)] -
    (1.0f/8.0f) * u_x[j*(nx+1) + (i+2)];
288         }
289 
290         // West face
291         if (Fw >= 0) {
292             phi_w = (3.0f/8.0f) * u_x[j*(nx+1) + i] + (6.0f/8.0f) * u_x[j*(nx+1) + (i-1)]
    -(1.0f/8.0f) * u_x[j*(nx+1) + (i-2)];
293         } else {
294             phi_w = (3.0f/8.0f) * u_x[j*(nx+1) + (i-1)] + (6.0f/8.0f) * u_x[j*(nx+1) + i]
    - (1.0f/8.0f) * u_x[j*(nx+1) + (i+1)];
295         }
296 
297         // North face
298         if (Fn >= 0) {
299             phi_n = (3.0f/8.0f) * u_x[(j+1)*(nx+1) + i] +(6.0f/8.0f) * u_x[j*(nx+1) + i] -
    (1.0f/8.0f) * u_x[(j-1)*(nx+1) + i];
300         } else {
301             phi_n = (3.0f/8.0f) * u_x[j*(nx+1) + i] +(6.0f/8.0f) * u_x[(j+1)*(nx+1) + i] -
    (1.0f/8.0f) * u_x[(j+2)*(nx+1) + i];
302         }
303 
304         // South face
305         if (Fs >= 0) {
306             phi_s = (3.0f/8.0f) * u_x[j*(nx+1) + i] + (6.0f/8.0f) * u_x[(j-1)*(nx+1) + i]
    - (1.0f/8.0f) * u_x[(j-2)*(nx+1) + i];
307         } else {
308             phi_s = (3.0f/8.0f) * u_x[(j-1)*(nx+1) + i] + (6.0f/8.0f) * u_x[j*(nx+1) + i]
    -(1.0f/8.0f) * u_x[(j+1)*(nx+1) + i];
309         }
310 
311         float convection_term = -(Fe * phi_e - Fw * phi_w + Fn * phi_n - Fs * phi_s) / (DX
    * DY);
312 
313         // 2. Calculate diffusion term using central difference
314         // Interpolate viscosity to u-cell faces
315         float mu_e = 0.5f * (mu[j*nx + i] + mu[j*nx + (i+1)]);
316         float mu_w = 0.5f * (mu[j*nx + (i-1)] + mu[j*nx + i]);
317         float mu_n = 0.5f * (mu[j*nx + i] + mu[(j+1)*nx + i]);
318         float mu_s = 0.5f * (mu[(j-1)*nx + i] + mu[j*nx + i]);
319 
320         float diffusion_term = (mu_e * (u_x[j*(nx+1) + (i+1)] - u_x[j*(nx+1) + i]) / DX -
    mu_w * (u_x[j*(nx+1) + i] - u_x[j*(nx+1) + (i-1)]) / DX) / DX + (mu_n * (u_x[(j+1)*(nx+1)
    + i] - u_x[j*(nx+1) + i]) / DY -mu_s * (u_x[j*(nx+1) + i] - u_x[(j-1)*(nx+1) + i]) / DY) /
    DY;
321 
322         // 3. Calculate pressure gradient (staggered grid - pressure at cell centers)
323         // For u_x at (i+1/2,j), we need pressure gradient between p[i,j] and p[i+1,j]
324         float pressure_gradient_term = (p[j*nx + i] - p[j*nx + (i-1)]) / DX;
325 
```

```
326         // 4. Update u_x using the momentum equation
327         u_x_new[j*(nx+1) + i] = u_x[j*(nx+1) + i] +dt * (-convection_term +
    diffusion_term/rho - pressure_gradient_term/rho);
328     }
329 }
330
331 //solver for y momentum equation
332 //Ymomentum equation solver
333 //solver for y momentum equation
334 //Ymomentum equation solver
335 __global__ void YMomentum_solver(float* u_x, float* u_y, float* p, float* v_new, float dt,
    float dx, float dy, float rho, float* mu, int nx, int ny) {
336     int i = blockIdx.x * blockDim.x + threadIdx.x;
337     int j = blockIdx.y * blockDim.y + threadIdx.y;
338     if (i >= 1 && i < nx-1 && j >= 1 && j < ny-1) {  // Fixed upper bound check for j
339         // For staggered grid, v is at (i,j+1/2)
340         int idx = i + j*nx;  // V_IDX(i, j, nx)
341
342         float dx_e = dx, dx_w = dx, dy_n = dy, dy_s = dy;
343
344         // Interpolate u-velocities to v-cell faces (east and west)
345         float ue = 0.25f * (u_x[i + j*(nx+1)] + u_x[(i+1) + j*(nx+1)] +u_x[i +
    (j-1)*(nx+1)] + u_x[(i+1) + (j-1)*(nx+1)]);
346         float Fe = ue * dy_n;
347         float uw = 0.25f * (u_x[(i-1) + j*(nx+1)] + u_x[i + j*(nx+1)] + u_x[(i-1) +
    (j-1)*(nx+1)] + u_x[i + (j-1)*(nx+1)]);
348         float Fw = uw * dy_s;
349
350         // Interpolate v-velocities to v-cell faces (north and south)
351         float vn = 0.5f * (u_y[i + j*nx] + u_y[i + (j+1)*nx]);
352         float Fn = vn * dx_e;
353         float vs = 0.5f * (u_y[i + (j-1)*nx] + u_y[i + j*nx]);
354         float Fs = vs * dx_w;
355
356         // QUICK scheme for face values with flow direction check
357         float phi_e, phi_w, phi_n, phi_s;
358
359         // East face
360         if (Fe >= 0) {
361             phi_e = (3.0f/8.0f) * u_y[(i+1) + j*nx] + (6.0f/8.0f) * u_y[i + j*nx] -
    (1.0f/8.0f) * u_y[(i-1) + j*nx];
362
363
364         } else {
365             phi_e = (3.0f/8.0f) * u_y[i + j*nx] + (6.0f/8.0f) * u_y[(i+1) + j*nx] - (1.0f/
    8.0f) * u_y[(i+2) + j*nx];
366         }
367
368         // West face
369         if (Fw >= 0) {
370             phi_w = (3.0f/8.0f) * u_y[i + j*nx] + (6.0f/8.0f) * u_y[(i-1) + j*nx] - (1.0f/
    8.0f) * u_y[(i-2) + j*nx];
371
372         } else {
373             phi_w = (3.0f/8.0f) * u_y[(i-1) + j*nx] + (6.0f/8.0f) * u_y[i + j*nx] - (1.0f/
    8.0f) * u_y[(i+1) + j*nx];
374         }
375
376         // North face
377         if (Fn >= 0) {
```

```
378             phi_n = (3.0f/8.0f) * u_y[i + (j+1)*nx] + (6.0f/8.0f) * u_y[i + j*nx] - (1.0f/
    8.0f) * u_y[i + (j-1)*nx];
379         } else {
380             phi_n = (3.0f/8.0f) * u_y[i + j*nx] +(6.0f/8.0f) * u_y[i + (j+1)*nx] -(1.0f/
    8.0f) * u_y[i + (j+2)*nx];
381         }
382 
383         // South face
384         if (Fs >= 0) {
385             phi_s = (3.0f/8.0f) * u_y[i + j*nx] +(6.0f/8.0f) * u_y[i + (j-1)*nx] -(1.0f/
    8.0f) * u_y[i + (j-2)*nx];
386         } else {
387             phi_s = (3.0f/8.0f) * u_y[i + (j-1)*nx] +(6.0f/8.0f) * u_y[i + j*nx] - (1.0f/
    8.0f) * u_y[i + (j+1)*nx];
388         }
389 
390         // Viscosity interpolation - corrected indexing for staggered grid
391         float mu_e = 0.5f * (mu[i + j*nx] + mu[(i+1) + j*nx]);
392         float mu_w = 0.5f * (mu[(i-1) + j*nx] + mu[i + j*nx]);
393         float mu_n = 0.5f * (mu[i + j*nx] + mu[i + (j+1)*nx]);
394         float mu_s = 0.5f * (mu[i + (j-1)*nx] + mu[i + j*nx]);
395 
396         float De = mu_e * dy_n / dx_e;
397         float Dw = mu_w * dy_s / dx_w;
398         float Dn = mu_n * dx_e / dy_n;
399         float Ds = mu_s * dx_w / dy_s;
400 
401         // Pressure gradient - corrected for v-velocity at (i,j+1/2)
402         float dp_dy = (p[i + j*nx] - p[i + (j-1)*nx]) / dy;
403 
404         float convection = Fe * phi_e - Fw * phi_w + Fn * phi_n - Fs * phi_s;
405         float diffusion = De * (u_y[(i+1) + j*nx] - u_y[i + j*nx])- Dw * (u_y[i + j*nx] -
    u_y[(i-1) + j*nx])+ Dn * (u_y[i + (j+1)*nx] - u_y[i + j*nx])- Ds * (u_y[i + j*nx] - u_y[i
    + (j-1)*nx]);
406 
407         v_new[idx] = u_y[idx] + dt * (-convection/(dx*dy) + diffusion/(dx*dy) - dp_dy/
    rho);
408     }
409 }
410 
411 
412 // SOLVER FOR PRESSURE CORRECTION STEP
413 // Pressure correction solver
414 //COMPUTING THE DIVERGENCE FOR BOTH U_NEW AND V_NEW
415 
416 __global__ void Divergence(float *u_x_new, float *v_new, float *divergence, int nx, int
    ny) {
417     int i = blockIdx.x * blockDim.x + threadIdx.x;
418     int j = blockIdx.y * blockDim.y + threadIdx.y;
419 
420     if (i >= 1 && i < nx-1 && j >= 1 && j < ny-1) {
421         // For staggered grid:
422         // u_x is stored at (i+1/2,j) - east/west cell faces
423         // v is stored at (i,j+1/2) - north/south cell faces
424 
425         // Calculate divergence using central difference
426         // u_x at east face (i+1/2,j) minus u_x at west face (i-1/2,j)
427         float du_dx = (u_x_new[j*(nx+1) + (i+1)] - u_x_new[j*(nx+1) + i]) / DX;
428 
429         // v at north face (i,j+1/2) minus v at south face (i,j-1/2)
```

```
        float dv_dy = (v_new[(j+1)*nx + i] - v_new[j*nx + i]) / DY;

        // Store divergence at cell center (i,j)
        divergence[j*nx + i] = du_dx + dv_dy;
    }
}

// Pressure correction solver
// Pressure Poisson equation solver using Gauss-Seidel with SOR
__global__ void PoissonSolverGS(float *divergence, float *p, float *p_new, int nx, int ny, float omega, float *residual) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    if (i >= 1 && i < nx-1 && j >= 1 && j < ny-1) {
        // Proper indexing for staggered grid (cell centers)
        int idx = j*nx + i;  // Row-major order

        float source = -rho * divergence[idx] / dt;
        float p_old = p[idx];

        // Gauss-Seidel iteration for pressure correction
        float p_gs = 0.25f * (
            p_new[j*nx + (i+1)] + p_new[j*nx + (i-1)] +
            p_new[(j+1)*nx + i] + p_new[(j-1)*nx + i] -
            DX * DY * source
        );

        // Apply SOR (Successive Over-Relaxation)
        p_new[idx] = (1.0f - omega) * p_old + omega * p_gs;

        // Calculate local residual for convergence check
        // Using atomicAdd can cause performance issues due to thread contention
        // Better to compute local residuals and reduce them later
        atomicAdd(residual, fabsf(p_new[idx] - p_old));
    }
}

// CONVERGENCE CHECK FOR Host-side wrapper for Gauss-Seidel iteration with convergence check
// Host-side wrapper for Gauss-Seidel iteration with convergence check
bool SolvePressurePoisson(float *d_divergence, float *d_p, float *d_p_new, int nx, int ny, float omega, float tolerance) {
    dim3 block(16, 16);
    dim3 grid((nx + block.x - 1) / block.x, (ny + block.y - 1) / block.y);

    float *d_residual;
    cudaMalloc((void**)&d_residual, sizeof(float));
    float h_residual;

    int max_iter = 1000;  // Maximum iterations

    for (int iter = 0; iter < max_iter; ++iter) {
        // Reset residual
        cudaMemset(d_residual, 0, sizeof(float));

        // Copy current pressure to p_new as initial guess
        cudaMemcpy(d_p_new, d_p, nx * ny * sizeof(float), cudaMemcpyDeviceToDevice);

        // Perform one Gauss-Seidel iteration
```

```
        PoissonSolverGS<<<grid, block>>>(d_divergence, d_p, d_p_new, nx, ny, omega, d_residual);
        cudaDeviceSynchronize();

        // Check convergence
        cudaMemcpy(&h_residual, d_residual, sizeof(float), cudaMemcpyDeviceToHost);
        h_residual /= (nx * ny);  // Normalize by grid size

        if (h_residual < tolerance) {
            cudaFree(d_residual);
            return true;  // Converged
        }

        // Update pressure for next iteration
        cudaMemcpy(d_p, d_p_new, nx * ny * sizeof(float), cudaMemcpyDeviceToDevice);
    }

    cudaFree(d_residual);
    return false;  // Did not converge within max_iter
}


// Correct velocities using pressure correction based on SIMPLE algorithm using staggered grid
//with time step implementation
__global__ void CorrectVelocities(float *u, float *v, float *u_star, float *v_star, float *p, int nx, int ny) {
    int i = blockIdx.x * blockDim.x + threadIdx.x;
    int j = blockIdx.y * blockDim.y + threadIdx.y;

    // Correct u-velocity (staggered at i+1/2, j)
    if (i >= 1 && i < nx && j >= 1 && j < ny-1) {
        // d-coefficient from momentum equation (simplified)
        float d_u = dt / rho;

        // Proper indexing for staggered grid
        int idx_u = j*(nx+1) + i;  // u at (i+1/2,j)

        // Pressure gradient between adjacent cell centers
        float dp_dx = (p[j*nx + i] - p[j*nx + (i-1)]) / DX;

        // Correct u-velocity using pressure gradient
        u[idx_u] = u_star[idx_u] - d_u * dp_dx;
    }

    // Correct v-velocity (staggered at i, j+1/2)
    if (i >= 1 && i < nx-1 && j >= 1 && j < ny) {
        // d-coefficient from momentum equation (simplified)
        float d_v = dt / rho;

        // Proper indexing for staggered grid
        int idx_v = j*nx + i;  // v at (i,j+1/2)

        // Pressure gradient between adjacent cell centers
        float dp_dy = (p[j*nx + i] - p[(j-1)*nx + i]) / DY;

        // Correct v-velocity using pressure gradient
        v[idx_v] = v_star[idx_v] - d_v * dp_dy;
    }
}
```

```
544
545 // Main temporal simulation loop
546 __host__ void runSimulation(float *d_u, float *d_v, float *d_p, float *d_visc, float
    *d_shear_rate, float *d_u_star, float *d_v_star, float *d_p_prime, float *d_divergence) {
547     // Simulation parameters
548     float physicalTime = 5.0f;        // Total physical time to simulate (seconds)
549     float currentTime = 0.0f;         // Current physical time
550     int timeStep = 0;                 // Current time step
551     float cfl = 1.0f;                 // Target CFL number
552     float maxResidual = 1.0f;         // Initial residual
553     float convergenceTolerance = 1e-5f; // Convergence criteria
554     int maxIterations = 10000;         // Maximum SIMPLE iterations per time step
555     int outputInterval = 100;         // Output every N time steps
556
557     // Ask user for simulation parameters
558     printf("Enter total physical time (default: 5.0s): ");
559     scanf("%f", &physicalTime);
560     printf("Enter output interval in time steps (default: 100): ");
561     scanf("%d", &outputInterval);
562
563     // Calculate adaptive time step based on CFL condition
564     float maxVelocity = U;  // Initial estimate
565     float adaptiveDt = cfl * DX / maxVelocity;
566     printf("Initial time step: %e seconds\n", adaptiveDt);
567
568     // Allocate host memory for monitoring
569     float *h_u = (float*)malloc(sizeof(float) * (NX+1) * NY);
570     float *h_v = (float*)malloc(sizeof(float) * NX * (NY+1));
571     float *h_p = (float*)malloc(sizeof(float) * NX * NY);
572     float *h_residual = (float*)malloc(sizeof(float));
573
574     // Execution configuration
575     dim3 block(16, 16);
576     dim3 grid((NX + block.x - 1) / block.x, (NY + block.y - 1) / block.y);
577
578     // Create output file
579     FILE *outfile = fopen("blood_flow_results.csv", "w");
580     fprintf(outfile, "Time,TimeStep,Iterations,MaxResidual,MaxVelocity,MaxPressure\n");
581
582     // Time advancement loop
583     while (currentTime < physicalTime) {
584         timeStep++;
585
586         // Store previous time step values for temporal discretization
587         cudaMemcpy(d_u_old, d_u, sizeof(float) * (NX+1) * NY, cudaMemcpyDeviceToDevice);
588         cudaMemcpy(d_v_old, d_v, sizeof(float) * NX * (NY+1), cudaMemcpyDeviceToDevice);
589
590         // SIMPLE algorithm iterations
591         int iterations = 0;
592         maxResidual = 1.0f;
593
594         while (iterations < maxIterations && maxResidual > convergenceTolerance) {
595             // Step 1: Momentum prediction
596             Xmomentum_solver<<<grid, block>>>(d_u, d_v, d_p, d_visc, d_u_star, NX, NY);
597            YMomentum_solver<<<grid, block>>>(d_u, d_v, d_p, d_v_star, adaptiveDt, DX, DY,
    rho, d_visc, NX, NY);
598
599             // Step 2: Calculate divergence
600             Divergence<<<grid, block>>>(d_u_star, d_v_star, d_divergence, NX, NY);
601
```

```
602             // Step 3: Solve pressure correction equation
603             float omega = 1.5f;  // SOR relaxation parameter
604             bool converged = SolvePressurePoisson(d_divergence, d_p, d_p_prime, NX, NY,
    omega, convergenceTolerance/10.0f);
605 
606             // Step 4: Correct velocities and pressure
607             CorrectVelocities<<<grid, block>>>(d_u, d_v, d_u_star, d_v_star, d_p, NX, NY);
608 
609             // Step 5: Update viscosity using Bird-Carreau model
610             Shear_rate<<<grid, block>>>(d_u, d_v, d_visc, d_shear_rate, NX, NY);
611 
612             // Calculate residual for convergence check
613             float *d_residual;
614             cudaMalloc((void**)&d_residual, sizeof(float));
615             cudaMemset(d_residual, 0, sizeof(float));
616 
617             // Calculate velocity residual (change between iterations)
618             CalculateResidual<<<grid, block>>>(d_u, d_u_old, d_v, d_v_old, d_residual, NX,
    NY);
619             cudaMemcpy(h_residual, d_residual, sizeof(float), cudaMemcpyDeviceToHost);
620             maxResidual = *h_residual;
621 
622             cudaFree(d_residual);
623             iterations++;
624 
625             // Store current values for next iteration
626             cudaMemcpy(d_u_old, d_u, sizeof(float) * (NX+1) * NY,
    cudaMemcpyDeviceToDevice);
627             cudaMemcpy(d_v_old, d_v, sizeof(float) * NX * (NY+1),
    cudaMemcpyDeviceToDevice);
628         }
629 
630         // Update current time
631         currentTime += adaptiveDt;
632 
633         // Calculate maximum velocity for adaptive time stepping
634         float *d_max_velocity;
635         cudaMalloc((void**)&d_max_velocity, sizeof(float));
636         cudaMemset(d_max_velocity, 0, sizeof(float));
637 
638         CalculateMaxVelocity<<<grid, block>>>(d_u, d_v, d_max_velocity, NX, NY);
639         float h_max_velocity;
640         cudaMemcpy(&h_max_velocity, d_max_velocity, sizeof(float),
    cudaMemcpyDeviceToHost);
641         cudaFree(d_max_velocity);
642 
643         // Update time step based on CFL condition
644         if (h_max_velocity > 0.01f) {  // Avoid division by very small values
645             adaptiveDt = cfl * DX / h_max_velocity;
646         }
647 
648         // Limit time step growth for stability
649         adaptiveDt = fminf(adaptiveDt, 1.5f * adaptiveDt);
650 
651         // Adjust final time step to exactly reach physicalTime
652         if (currentTime + adaptiveDt > physicalTime) {
653             adaptiveDt = physicalTime - currentTime;
654         }
655 
656         // Output results at specified intervals
```

```
657         if (timeStep % outputInterval == 0 || currentTime >= physicalTime) {
658             printf("Time: %.4f s, Step: %d, Iterations: %d, Residual: %.6e, Max Velocity:
    %.4f m/s\n",
659                    currentTime, timeStep, iterations, maxResidual, h_max_velocity);
660
661             // Copy data back to host for output
662             cudaMemcpy(h_u, d_u, sizeof(float) * (NX+1) * NY, cudaMemcpyDeviceToHost);
663             cudaMemcpy(h_v, d_v, sizeof(float) * NX * (NY+1), cudaMemcpyDeviceToHost);
664             cudaMemcpy(h_p, d_p, sizeof(float) * NX * NY, cudaMemcpyDeviceToHost);
665
666             // Calculate maximum pressure
667             float maxPressure = 0.0f;
668             for (int i = 0; i < NX*NY; i++) {
669                 if (fabs(h_p[i]) > maxPressure) maxPressure = fabs(h_p[i]);
670             }
671
672             // Write to output file
673             fprintf(outfile, "%.6f,%d,%d,%.6e,%.6f,%.6f\n",
674                     currentTime, timeStep, iterations, maxResidual, h_max_velocity,
    maxPressure);
675
676             // Write field data to VTK file for visualization
677             char filename[256];
678             sprintf(filename, "blood_flow_t%.4f.vtk", currentTime);
679             writeVTKFile(filename, h_u, h_v, h_p, NX, NY, DX, DY);
680         }
681     }
682
683     // Close output file
684     fclose(outfile);
685
686     // Free host memory
687     free(h_u);
688     free(h_v);
689     free(h_p);
690     free(h_visc);
691     free(h_shear_rate);
692
693     printf("Simulation completed: %.4f seconds simulated in %d time steps\n", currentTime,
    timeStep);
694 }
695
696 // Function to save data in VTK format for ParaView visualization
697 // Function to save data in VTK format for ParaView visualization
698 void saveVTKFile(float *u, float *v, float *p, float *mu, float *shear_rate, int nx, int
    ny, int timestep) {
699     char filename[256];
700     sprintf(filename, "blood_flow_%04d.vtk", timestep);
701     FILE *fp = fopen(filename, "w");
702     if (fp == NULL) {
703         printf("Error opening file for VTK output\n");
704         return;
705     }
706
707     // VTK header
708     fprintf(fp, "# vtk DataFile Version 3.0\n");
709     fprintf(fp, "LAD Artery Blood Flow Simulation\n");
710     fprintf(fp, "ASCII\n");
711     fprintf(fp, "DATASET STRUCTURED_POINTS\n");
712     fprintf(fp, "DIMENSIONS %d %d 1\n", nx, ny);
```

```
    fprintf(fp, "ORIGIN 0 0 0\n");
    fprintf(fp, "SPACING %f %f 1.0\n", DX, DY);
    fprintf(fp, "POINT_DATA %d\n", nx * ny);

    fprintf(fp, "SCALARS u_velocity float 1\nLOOKUP_TABLE default\n");
    for (int j = 0; j < ny; j++)
        for (int i = 0; i < nx; i++)
            fprintf(fp, "%f\n", u[i*ny + j]);
    fprintf(fp, "SCALARS v_velocity float 1\nLOOKUP_TABLE default\n");
    for (int j = 0; j < ny; j++)
        for (int i = 0; i < nx; i++)
            fprintf(fp, "%f\n", v[i*ny + j]);
    fprintf(fp, "SCALARS pressure float 1\nLOOKUP_TABLE default\n");
    for (int j = 0; j < ny; j++)
        for (int i = 0; i < nx; i++)
            fprintf(fp, "%f\n", p[i*ny + j]);
    fprintf(fp, "SCALARS viscosity float 1\nLOOKUP_TABLE default\n");
    for (int j = 0; j < ny; j++)
        for (int i = 0; i < nx; i++)
            fprintf(fp, "%f\n", mu[i*ny + j]);
    fprintf(fp, "SCALARS shear_rate float 1\nLOOKUP_TABLE default\n");
    for (int j = 0; j < ny; j++)
        for (int i = 0; i < nx; i++)
            fprintf(fp, "%f\n", shear_rate[i*ny + j]);
    fprintf(fp, "VECTORS velocity float\n");
    for (int j = 0; j < ny; j++)
        for (int i = 0; i < nx; i++)
            fprintf(fp, "%f %f 0.0\n", u[i*ny + j], v[i*ny + j]);
    fclose(fp);
    printf("Saved VTK file: %s\n", filename);
}

int main() {
    // Host arrays
    float *h_u, *h_v, *h_p, *h_visc, *h_shear_rate;
    // Device arrays
    float *d_u, *d_v, *d_p, *d_visc, *d_shear_rate;
    float *d_u_star, *d_v_star, *d_p_prime, *d_divergence;
    float *d_u_old, *d_v_old;

    // Allocate host memory
    h_u = (float*)malloc(sizeof(float) * (NX+1) * NY);
    h_v = (float*)malloc(sizeof(float) * NX * (NY+1));
    h_p = (float*)malloc(sizeof(float) * NX * NY);
    h_visc = (float*)malloc(sizeof(float) * NX * NY);
    h_shear_rate = (float*)malloc(sizeof(float) * NX * NY);

    // Allocate device memory
    cudaMalloc((void**)&d_u, sizeof(float) * (NX+1) * NY);
    cudaMalloc((void**)&d_v, sizeof(float) * NX * (NY+1));
    cudaMalloc((void**)&d_p, sizeof(float) * NX * NY);
    cudaMalloc((void**)&d_visc, sizeof(float) * NX * NY);
    cudaMalloc((void**)&d_shear_rate, sizeof(float) * NX * NY);
    cudaMalloc((void**)&d_u_star, sizeof(float) * (NX+1) * NY);
    cudaMalloc((void**)&d_v_star, sizeof(float) * NX * (NY+1));
    cudaMalloc((void**)&d_p_prime, sizeof(float) * NX * NY);
    cudaMalloc((void**)&d_divergence, sizeof(float) * NX * NY);
    cudaMalloc((void**)&d_u_old, sizeof(float) * (NX+1) * NY);
    cudaMalloc((void**)&d_v_old, sizeof(float) * NX * (NY+1));

```

```
    // Initialize host arrays
    for (int j = 0; j < NY; j++) {
        for (int i = 0; i < NX; i++) {
            int idx = i + j*NX;
            h_p[idx] = del_P * (1.0f - (float)i/NX);
            h_visc[idx] = mu0;
            h_shear_rate[idx] = 0.0f;
        }
    }
    for (int j = 0; j < NY; j++) {
        for (int i = 0; i <= NX; i++) {
            int idx = i + j*(NX+1);
            h_u[idx] = (i == 0) ? U : 0.0f;
        }
    }
    for (int j = 0; j <= NY; j++) {
        for (int i = 0; i < NX; i++) {
            int idx = i + j*NX;
            h_v[idx] = 0.0f;
        }
    }

    // Copy initial data from host to device
    cudaMemcpy(d_p, h_p, sizeof(float)*NX*NY, cudaMemcpyHostToDevice);
    cudaMemcpy(d_visc, h_visc, sizeof(float)*NX*NY, cudaMemcpyHostToDevice);
    cudaMemcpy(d_shear_rate, h_shear_rate, sizeof(float)*NX*NY, cudaMemcpyHostToDevice);
    cudaMemcpy(d_u, h_u, sizeof(float)*(NX+1)*NY, cudaMemcpyHostToDevice);
    cudaMemcpy(d_v, h_v, sizeof(float)*NX*(NY+1), cudaMemcpyHostToDevice);

    // Set execution configuration
    dim3 blockSize(16, 16);
    dim3 gridSize((NX + blockSize.x - 1) / blockSize.x, (NY + blockSize.y - 1) /
blockSize.y);

    // Simulation parameters
    float physicalTime = 5.0f;
    float currentTime = 0.0f;
    int timeStep = 0;
    float cfl = 0.5f;
    float maxResidual = 1.0f;
    float convergenceTolerance = 1e-5f;
    int maxIterations = 1000;
    int outputInterval = 100;

    printf("Enter total physical time (default: 5.0s): ");
    scanf("%f", &physicalTime);
    printf("Enter output interval in time steps (default: 100): ");
    scanf("%d", &outputInterval);

    float maxVelocity = U;
    float adaptiveDt = cfl * DX / maxVelocity;
    printf("Initial time step: %e seconds\n", adaptiveDt);

    FILE *outfile = fopen("blood_flow_results.csv", "w");
    fprintf(outfile, "Time,TimeStep,Iterations,MaxResidual,MaxVelocity,MaxPressure\n");

    while (currentTime < physicalTime) {
        timeStep++;
        cudaMemcpy(d_u_old, d_u, sizeof(float) * (NX+1) * NY, cudaMemcpyDeviceToDevice);
        cudaMemcpy(d_v_old, d_v, sizeof(float) * NX * (NY+1), cudaMemcpyDeviceToDevice);
```

```
832
833         int iterations = 0;
834         maxResidual = 1.0f;
835
836         while (iterations < maxIterations && maxResidual > convergenceTolerance) {
837             ApplyBoundaryConditions<<<gridSize, blockSize>>>(d_u, d_v, d_p, 0.0f, NX, NY);
838             Xmomentum_solver<<<gridSize, blockSize>>>(d_u, d_v, d_p, d_visc, d_u_star, NX,
    NY);
839             YMomentum_solver<<<gridSize, blockSize>>>(d_u, d_v, d_p, d_v_star, adaptiveDt,
    DX, DY, rho, d_visc, NX, NY);
840             Divergence<<<gridSize, blockSize>>>(d_u_star, d_v_star, d_divergence, NX, NY);
841
842             float omega = 1.5f;
843             bool converged = SolvePressurePoisson(d_divergence, d_p, d_p_prime, NX, NY,
    omega, convergenceTolerance/10.0f);
844
845             CorrectVelocities<<<gridSize, blockSize>>>(d_u, d_v, d_u_star, d_v_star, d_p,
    NX, NY);
846             Shear_rate<<<gridSize, blockSize>>>(d_u, d_v, d_visc, d_shear_rate, NX, NY);
847
848             float *d_residual;
849             cudaMalloc((void**)&d_residual, sizeof(float));
850             cudaMemset(d_residual, 0, sizeof(float));
851             CalculateResidual<<<gridSize, blockSize>>>(d_u, d_u_old, d_v, d_v_old,
    d_residual, NX, NY);
852             cudaMemcpy(&maxResidual, d_residual, sizeof(float), cudaMemcpyDeviceToHost);
853             cudaFree(d_residual);
854
855             iterations++;
856             cudaMemcpy(d_u_old, d_u, sizeof(float) * (NX+1) * NY,
    cudaMemcpyDeviceToDevice);
857             cudaMemcpy(d_v_old, d_v, sizeof(float) * NX * (NY+1),
    cudaMemcpyDeviceToDevice);
858         }
859
860         currentTime += adaptiveDt;
861
862         float *d_max_velocity;
863         cudaMalloc((void**)&d_max_velocity, sizeof(float));
864         cudaMemset(d_max_velocity, 0, sizeof(float));
865         CalculateMaxVelocity<<<gridSize, blockSize>>>(d_u, d_v, d_max_velocity, NX, NY);
866         float h_max_velocity;
867         cudaMemcpy(&h_max_velocity, d_max_velocity, sizeof(float),
    cudaMemcpyDeviceToHost);
868         cudaFree(d_max_velocity);
869
870         if (h_max_velocity > 0.01f) {
871             adaptiveDt = cfl * DX / h_max_velocity;
872         }
873         adaptiveDt = fminf(adaptiveDt, 1.5f * adaptiveDt);
874         if (currentTime + adaptiveDt > physicalTime) {
875             adaptiveDt = physicalTime - currentTime;
876         }
877
878         if (timeStep % outputInterval == 0 || currentTime >= physicalTime) {
879             printf("Time: %.4f s, Step: %d, Iterations: %d, Residual: %.6e, Max Velocity:
    %.4f m/s\n",
880                    currentTime, timeStep, iterations, maxResidual, h_max_velocity);
881
882             cudaMemcpy(h_u, d_u, sizeof(float) * (NX+1) * NY, cudaMemcpyDeviceToHost);
```

```
883             cudaMemcpy(h_v, d_v, sizeof(float) * NX * (NY+1), cudaMemcpyDeviceToHost);
884             cudaMemcpy(h_p, d_p, sizeof(float) * NX * NY, cudaMemcpyDeviceToHost);
885 
886             float maxPressure = 0.0f;
887             for (int i = 0; i < NX*NY; i++) {
888                 if (fabs(h_p[i]) > maxPressure) maxPressure = fabs(h_p[i]);
889             }
890             fprintf(outfile, "%.6f,%d,%d,%.6e,%.6f,%.6f\n",
891                     currentTime, timeStep, iterations, maxResidual, h_max_velocity,
    maxPressure);
892 
893             saveVTKFile(h_u, h_v, h_p, h_visc, h_shear_rate, NX, NY, timeStep);
894         }
895     }
896 
897     fclose(outfile);
898     free(h_u); free(h_v); free(h_p); free(h_visc); free(h_shear_rate);
899     cudaFree(d_u); cudaFree(d_v); cudaFree(d_p); cudaFree(d_visc); cudaFree(d_shear_rate);
900     cudaFree(d_u_star); cudaFree(d_v_star); cudaFree(d_p_prime); cudaFree(d_divergence);
901     cudaFree(d_u_old); cudaFree(d_v_old);
902 
903     printf("Simulation completed.\n");
904     return 0;
905 }
```